

Artificial Intelligence (New) — Exam Answer Sheet

Pokhara University | Master's, Semester Fall 2026 | ME Computer

Q1(a) What is Simulated Annealing in local search? Explain its working mechanism and how it helps in escaping local minima. (7 marks) — Unit 1

Definition: Simulated Annealing (SA) is a probabilistic local search algorithm inspired by the annealing process in metallurgy, where a metal is heated and then slowly cooled to remove defects and reach a low-energy, stable crystalline structure. In AI, SA is used to find a good approximate solution to an optimization problem by allowing occasional "bad" moves, which helps it avoid getting stuck in local optima.

Working Mechanism:

1. Start with a random initial state, and set a high initial "temperature" T .
2. Pick a random neighboring state.
3. Calculate the change in energy (or cost/objective value): $\Delta E = E(\text{neighbor}) - E(\text{current})$.
4. If ΔE is negative (improvement): Accept the new state.
5. If ΔE is positive (worse): Accept it anyway with probability: $P = e^{(-\Delta E / T)}$
6. Gradually decrease T according to a cooling schedule (e.g., $T = T \times \alpha$, where α is close to but less than 1).
7. Repeat steps 2–6 until T approaches zero or a stopping criterion is met.

Pseudocode:

```
current = initial_state
T = T_initial
while T > T_min:
    next = random_neighbor(current)
    ΔE = value(next) - value(current)
    if ΔE > 0:
        current = next
    else:
        if random(0,1) < e^(ΔE/T):
            current = next
    T = T * cooling_rate
return current
```

How it escapes local minima:

- Standard hill climbing only accepts moves that improve the solution, so it gets trapped as soon as it reaches a local optimum (all neighbors are worse).
- SA allows *occasional* moves to worse states, controlled by temperature T :
 - **High T (early stage):** High probability of accepting worse moves → broad exploration of the search space.
 - **Low T (later stage):** Very low probability of accepting worse moves → the algorithm settles down (exploitation) near a good solution.
- This exploration-then-exploitation balance lets SA "jump over" the walls of a local minimum's basin of attraction to potentially reach the global minimum.

Application example: Used in the Traveling Salesman Problem (TSP), VLSI circuit design, and job-shop scheduling.

Q1(b) What is Monte Carlo Tree Search (MCTS)? Explain its phases and discuss its application with example. (8 marks) — Unit 1

Definition: Monte Carlo Tree Search (MCTS) is a heuristic search algorithm for decision-making, especially in game playing, that builds a search tree incrementally and uses random sampling (Monte Carlo simulations) to estimate the value of moves, rather than exhaustively searching the entire game tree.

Four Phases of MCTS:

1. **Selection:** Starting from the root node, the algorithm recursively selects child nodes using a selection policy (commonly **UCB1** — Upper Confidence Bound) until it reaches a leaf node that is not fully expanded.

UCB1 formula:

$$UCB1 = (w_i / n_i) + c * \sqrt{\ln(N) / n_i}$$

where w_i = wins of node i , n_i = visits to node i , N = total visits to parent, c = exploration constant.

2. **Expansion:** If the selected leaf node is not a terminal state, one or more child nodes are added to the tree representing possible next moves.
3. **Simulation (Rollout):** From the newly expanded node, the algorithm plays out a random (or lightly-guided) sequence of moves until reaching a terminal state (win/loss/draw), without adding these to the tree.
4. **Backpropagation:** The result of the simulation is propagated back up through all the nodes visited in this iteration, updating their visit counts (n) and win statistics (w).

Diagram (describe as tree in exam):

```

[Root]
Selection ↓
[Node A]---[Node B]
Expansion ↓
[New Node]
Simulation ↓ (random payout to terminal state)
[Result: Win/Loss]
Backpropagation ↑ (update stats up to root)

```

This 4-step cycle repeats thousands of times within a time/computation budget. Finally, the child of the root with the highest visit count (or win rate) is chosen as the best move.

Application Example: MCTS is famously used in **AlphaGo/AlphaZero** to play the game of Go, where the search space ($\sim 10^{170}$ states) is too large for traditional minimax search. It's also used in general game-playing AI, and modern chess engines.

Q2(a) What are Hidden Markov Models (HMMs)? Explain their structure and how they are used for probabilistic reasoning over time with a suitable example. (7+8 marks) — Unit 2

Definition: A Hidden Markov Model (HMM) is a statistical model used to represent systems that evolve over time, where the actual states of the system are **hidden** (not directly observable), but we can observe outputs (evidence) that are probabilistically related to those hidden states.

Structure/Components of HMM:

1. **States (S):** A finite set of hidden states $\{S_1, S_2, \dots, S_n\}$ the system can be in.
2. **Observations (O):** A finite set of observable symbols/evidence emitted by the hidden states.
3. **Transition Probability Matrix (A):** $P(S_{i+1} | S_i)$ — probability of moving from one hidden state to another.
4. **Emission/Observation Probability Matrix (B):** $P(O_t | S_t)$ — probability of observing O_t given the system is in hidden state S_t .
5. **Initial State Distribution (π):** Probability distribution over the starting state.

Key Assumptions:

- **Markov Assumption:** The current state depends only on the previous state (first-order Markov chain).
- **Sensor/Output Independence:** The current observation depends only on the current hidden state.

Diagram description:

Hidden states: $S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow S_4$ (connected by transition probabilities A)
 $\downarrow \quad \downarrow \quad \downarrow \quad \downarrow$ (emission probabilities B)
 Observations: $O_1 \quad O_2 \quad O_3 \quad O_4$

How HMMs support probabilistic reasoning over time: HMMs allow us to answer key inference questions:

- **Filtering:** $P(\text{current state} | \text{observations so far})$ — used in Forward algorithm.
- **Prediction:** $P(\text{future state} | \text{observations so far})$.
- **Smoothing:** $P(\text{past state} | \text{all observations, past and future})$ — Forward-Backward algorithm.
- **Most Likely Explanation:** Finding the most probable sequence of hidden states given observations — solved using the **Viterbi algorithm**.

Example: Classic "Umbrella World": A security guard cannot see the weather (hidden state: Rain or Sunny) but observes whether their colleague brings an umbrella (observation: Umbrella or No Umbrella) each day. Using transition probabilities (Rain→Rain, Rain→Sunny, etc.) and emission probabilities ($P(\text{Umbrella}|\text{Rain})$, $P(\text{Umbrella}|\text{Sunny})$), the HMM lets the guard infer the most probable weather sequence purely from umbrella observations. This same structure is used in **speech recognition** (hidden = phonemes, observed = audio signal) and **POS tagging** in NLP.

Q2(a) — OR — What is Ontological Engineering in AI? Explain its process and significance in knowledge representation with an example. (Unit 2)

Definition: Ontological Engineering is the process of designing and building ontologies — formal, explicit specifications of a shared conceptualization of a domain — including the concepts (classes), their properties, relationships, and constraints, so that knowledge can be represented and reasoned about consistently by AI systems.

Process of Ontological Engineering:

1. **Domain and Scope Identification:** Define the boundaries of the domain to be modeled (e.g., "medical diagnosis" or "university system").
2. **Consider Reuse:** Check if existing ontologies (e.g., WordNet, SNOMED-CT) can be reused or extended.
3. **Enumerate Important Terms:** List key concepts and terms relevant to the domain.
4. **Define Classes and Class Hierarchy:** Organize concepts into a taxonomy (is-a relationships), e.g., Vehicle → Car → SUV.
5. **Define Properties (Slots) of Classes:** Attributes and relationships between classes (e.g., "Car has-a Engine").
6. **Define Facets/Constraints:** Value restrictions, cardinality, and other constraints on properties.
7. **Create Instances:** Populate the ontology with actual individual objects belonging to the classes.

Significance in Knowledge Representation:

- Provides a **shared vocabulary** so different AI systems/agents can interoperate and communicate meaningfully.

- Enables **automated reasoning** (inference) — e.g., if "Dog is-a Mammal" and "Mammal is-a Animal", the system infers "Dog is-a Animal".
- Reduces ambiguity, since concepts and their relations are explicitly and formally defined.
- Forms the backbone of the **Semantic Web** and expert systems.
- Supports reusability and modularity of knowledge across applications.

Example: In a university ontology: classes could include *Person* $\rightarrow \{Student, Teacher\}$, *Course*, *Department*. Properties: "Student enrolledIn Course", "Teacher teaches Course". This structured representation lets an AI system automatically answer queries like "Which teachers teach courses that Student X is enrolled in?" through inference over the ontology.

Q2(b) With given sentences prove 'Rajesh is busy' using resolution refutation. (8 marks) — Unit 2

Given statements:

1. Rajesh is a megastar. $\rightarrow \text{Megastar}(\text{Rajesh})$
2. Megastar needs more money. $\rightarrow \forall x \text{ Megastar}(x) \rightarrow \text{NeedsMoney}(x)$
3. If you need more money then you need to be busy. $\rightarrow \forall x \text{ NeedsMoney}(x) \rightarrow \text{Busy}(x)$

Goal: Prove $\text{Busy}(\text{Rajesh})$

Step 1: Convert to First-Order Logic (FOL)

1. $\text{Megastar}(\text{Rajesh})$
2. $\forall x \neg \text{Megastar}(x) \vee \text{NeedsMoney}(x)$
3. $\forall x \neg \text{NeedsMoney}(x) \vee \text{Busy}(x)$

Step 2: Negate the Goal (for refutation)

4. $\neg \text{Busy}(\text{Rajesh})$

Step 3: Convert to Clausal (CNF) Form

- C1: $\text{Megastar}(\text{Rajesh})$
- C2: $\neg \text{Megastar}(x) \vee \text{NeedsMoney}(x)$
- C3: $\neg \text{NeedsMoney}(x) \vee \text{Busy}(x)$
- C4: $\neg \text{Busy}(\text{Rajesh})$

Step 4: Apply Resolution

- **Resolve C1 and C2** (unify $x = \text{Rajesh}$): $\text{Megastar}(\text{Rajesh}) + \neg \text{Megastar}(\text{Rajesh}) \vee \text{NeedsMoney}(\text{Rajesh}) \rightarrow \text{C5: } \text{NeedsMoney}(\text{Rajesh})$
- **Resolve C5 and C3** (unify $x = \text{Rajesh}$): $\text{NeedsMoney}(\text{Rajesh}) + \neg \text{NeedsMoney}(\text{Rajesh}) \vee \text{Busy}(\text{Rajesh}) \rightarrow \text{C6: } \text{Busy}(\text{Rajesh})$
- **Resolve C6 and C4:** $\text{Busy}(\text{Rajesh}) + \neg \text{Busy}(\text{Rajesh}) \rightarrow \text{C7: NIL (empty clause / contradiction)}$

Conclusion: Since resolving the clauses (including the negated goal) leads to an empty clause (contradiction), this proves by **refutation** that the original goal $\text{Busy}(\text{Rajesh})$ logically follows from the given premises. Hence, "Rajesh is busy" is proved.



Q3(a) Differentiate between hierarchical planning and probabilistic planning. (7 marks) — Unit 3

Basis	Hierarchical Planning	Probabilistic Planning
Definition	Decomposes a complex task into a hierarchy of simpler sub-tasks/sub-goals using abstraction levels	Plans under uncertainty, where actions have probabilistic (non-deterministic) outcomes
Environment	Assumes deterministic environment (outcomes are certain)	Handles stochastic/uncertain environments
Representation	Uses task networks, HTN (Hierarchical Task Networks) with high-level and primitive actions	Uses Markov Decision Processes (MDPs), Partially Observable MDPs
Focus	Reducing complexity via abstraction (divide and conquer)	Handling risk and uncertainty in outcomes
Solution Output	A structured plan/sequence of sub-tasks	A policy — a mapping from states to actions, since exact outcomes aren't known in advance
Example	Planning "Build a House" → decomposed into "Lay Foundation" → "Build Walls" → "Roofing"	Robot navigation where movement actions may fail or drift due to sensor/actuator noise
Computation	Generally more efficient due to abstraction reducing search space	Computationally more expensive due to need to consider probability distributions over outcomes

Q3(b) What is a Bandit Problem in AI? Explain the exploration vs. exploitation trade-off with an example. (8 marks) — Unit 3

Definition: The Multi-Armed Bandit Problem is a classic problem in decision theory and reinforcement learning where an agent must repeatedly choose among several options ("arms" of a slot machine), each giving an unknown, probabilistic reward, with the goal of maximizing total reward over time.

The name comes from a "one-armed bandit" (slot machine); a "multi-armed bandit" has several levers, each with a different unknown payout probability.

Exploration vs. Exploitation Trade-off:

- **Exploitation:** Choosing the arm/action that has given the best results so far (based on current knowledge), to maximize immediate reward.
- **Exploration:** Trying other, less-certain arms/actions to gather more information, which might reveal a better option in the long run.

The trade-off arises because:

- Pure exploitation risks getting stuck with a sub-optimal choice if the best arm hasn't been discovered yet.
- Pure exploration wastes reward by trying bad options repeatedly instead of using known good ones.

Common Strategies to Balance the Trade-off:

1. **ϵ -greedy:** With probability ϵ , choose a random arm (explore); otherwise choose the best-known arm (exploit).
2. **Softmax action selection:** Choose actions probabilistically, weighted by their estimated value.
3. **Upper Confidence Bound (UCB):** Choose the arm that has the highest upper confidence bound on its estimated reward — balances trying uncertain-but-promising arms.

Example: Consider an online ad-recommendation system with 3 different ad banners (arms). Initially the system doesn't know the click-through rate (CTR) of each. It must:

- **Explore** by showing all three ads to gather CTR data,
- **Exploit** by increasingly showing the ad with the highest observed CTR to maximize clicks.

This same trade-off appears in clinical trials (testing different treatments), reinforcement learning agents choosing actions, and recommendation systems (Netflix, YouTube).

Q4(a) Explain with examples how CSP domains can be finite, infinite, and continuous, and analyze how domain type impacts CSP solving techniques. (7 marks) — Unit 4

Definition (CSP): A Constraint Satisfaction Problem (CSP) is defined by a set of variables, each with a domain of possible values, and a set of constraints specifying allowable combinations of values.

1. Finite Domains:

- Each variable can take a value from a limited, countable set.
- **Example:** Map coloring problem — each region's domain = {Red, Green, Blue}. Sudoku — each cell's domain = {1, 2, ..., 9}.
- **Solving techniques:** Backtracking search, constraint propagation (arc consistency, AC-3), forward checking work efficiently since the search space is bounded and enumerable.

2. Infinite Domains (Discrete but unbounded):

- The domain has infinitely many discrete values, e.g., the set of all integers.
- **Example:** Job-scheduling problems where start times can be any integer (representing minutes/hours), such as "Task A must start at least 2 hours after Task B".
- **Solving techniques:** Cannot use simple enumeration; need a **constraint language** (e.g., linear inequalities) to describe constraints symbolically. Linear programming and specialized algebraic solvers are used instead of brute-force search.

3. Continuous Domains:

- Variables can take any real-numbered value within a range.
- **Example:** Scheduling problems in operations research where a task's start and end time are real numbers (e.g., start at 3.5 hours), or robot motion planning with continuous coordinates.
- **Solving techniques:** Discrete search methods don't apply directly. Instead, **Linear Programming (LP)** is used for problems with linear constraints, allowing solutions to be found in polynomial time. For nonlinear constraints, optimization techniques like gradient-based methods are used.

Impact of Domain Type on Solving:

Domain Type	Search Space	Preferred Techniques
Finite	Bounded, enumerable	Backtracking, AC-3, forward checking
Infinite (discrete)	Unbounded but countable	Symbolic constraint languages, specialized solvers
Continuous	Uncountably infinite	Linear/nonlinear programming, gradient methods

As domain size/type grows in complexity, exhaustive search becomes infeasible, so solving techniques must shift from combinatorial search toward mathematical/algebraic optimization methods.

Q4(b) What do you mean by backtracking? Explain the use of backtracking with example. (8 marks) — Unit 4

Definition: Backtracking is a systematic, depth-first search algorithm for solving CSPs that incrementally builds candidate solutions and abandons ("backtracks from") a candidate as soon as it determines the candidate cannot possibly lead to a valid solution.

Working Mechanism:

1. Choose an unassigned variable.
2. Try assigning it a value from its domain (that is consistent with constraints so far).
3. If the assignment satisfies all constraints so far, move to the next variable (recurse).
4. If no value works for the current variable, **backtrack** — go back to the previous variable and try a different value.
5. Repeat until either a complete, consistent assignment is found (solution) or all possibilities are exhausted (no solution).

Pseudocode:

```
function BACKTRACK(assignment, CSP):
    if assignment is complete: return assignment
    var = SELECT-UNASSIGNED-VARIABLE(CSP)
    for value in ORDER-DOMAIN-VALUES(var, CSP):
        if value is consistent with assignment:
            add {var = value} to assignment
            result = BACKTRACK(assignment, CSP)
            if result ≠ failure: return result
            remove {var = value} from assignment
    return failure
```

Example — Map Coloring Problem: Color regions A, B, C (where A-B and B-C are adjacent) using colors {Red, Green} such that no two adjacent regions share a color.

1. Assign A = Red. Consistent so far.
2. Assign B = Red → **conflict** (A and B adjacent, same color) → backtrack, try B = Green. Consistent.
3. Assign C = Red → conflict with B? No, B-C adjacent and B=Green, C=Red is fine → Consistent.
4. Solution found: A=Red, B=Green, C=Red.

If at any point no color works for a region, the algorithm backtracks to the previous region and tries an alternate color, rather than restarting from scratch — making it far more efficient than plain brute-force generate-and-test. Backtracking is also enhanced with techniques like **forward checking** and **constraint propagation** to detect failures earlier and prune the search tree.

Q5(a) Explain the working of convolution neural network with appropriate figures and concepts. (7 marks) — Unit 5

Definition: A Convolutional Neural Network (CNN) is a deep learning architecture specially designed for processing grid-like data such as images, using convolution operations to automatically learn spatial hierarchies of features.

Architecture / Layers:

```
Input Image → [Convolution Layer] → [ReLU Activation] → [Pooling Layer]
              → (repeat above block) → [Flatten] → [Fully Connected Layer] → [Output/Softmax]
```

1. Convolution Layer:



- A small matrix called a **filter/kernel** (e.g., 3×3) slides ("convolves") over the input image.
- At each position, an element-wise multiplication and sum is performed, producing a **feature map**.
- Different filters detect different features: edges, corners, textures, etc.
- Key parameters: **stride** (step size of filter movement) and **padding** (adding border pixels to control output size).

2. Activation Function (ReLU):

- Applies $f(x) = \max(0, x)$ element-wise to introduce non-linearity, allowing the network to learn complex patterns.

3. Pooling Layer (Downsampling):

- Reduces the spatial dimensions of feature maps while retaining important information.
- **Max Pooling:** Takes the maximum value in each region (e.g., 2×2 window).
- Reduces computation and helps achieve translation invariance.

4. Flatten + Fully Connected (Dense) Layer:

- The final feature maps are flattened into a 1D vector.
- Passed through one or more fully connected layers that combine features for final classification.

5. Output Layer:

- Typically uses **Softmax** activation for multi-class classification, producing class probabilities.

How it works end-to-end: Early convolution layers learn low-level features (edges, colors), middle layers combine these into shapes/textures, and deeper layers learn high-level, task-specific features (e.g., "cat face", "wheel of car"). The network is trained end-to-end using backpropagation and gradient descent, adjusting filter weights to minimize classification error.

Example application: Image classification (e.g., classifying handwritten digits in MNIST, object detection in self-driving cars).

Q5(b) Explain the transformer model architecture. (8 marks) — Unit 5

Definition: The Transformer is a deep learning architecture introduced in "Attention is All You Need" (2017) that relies entirely on **attention mechanisms** (no recurrence or convolution) to model relationships between elements in a sequence, enabling highly parallel training and capturing long-range dependencies effectively.

Overall Structure: Encoder-Decoder architecture (each made of stacked identical layers, typically 6).

Key Components:

1. Input Embedding + Positional Encoding:

- Words/tokens are converted into dense vector embeddings.
- Since Transformers have no recurrence (no inherent notion of order), **positional encoding** (sinusoidal functions) is added to embeddings to inject information about token position in the sequence.

2. Self-Attention Mechanism (Scaled Dot-Product Attention):

- For each token, three vectors are derived: **Query (Q)**, **Key (K)**, **Value (V)**.
- Attention score is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

- This allows each word to "attend to" (weigh the importance of) every other word in the sequence, capturing contextual relationships regardless of distance.

3. Multi-Head Attention:

- Instead of a single attention function, multiple attention "heads" run in parallel, each learning to focus on different types of relationships (e.g., syntactic vs semantic), and their outputs are concatenated and linearly transformed.

4. Feed-Forward Neural Network:

- Each position's output from attention is passed through an identical fully connected feed-forward network (applied independently to each position).

5. Add & Norm (Residual Connections + Layer Normalization):

- Residual (skip) connections around each sub-layer, followed by layer normalization, help stabilize training and allow deeper networks.

6. Encoder:

- Stack of layers, each with: Multi-Head Self-Attention → Add & Norm → Feed-Forward → Add & Norm.
- Encodes the input sequence into a contextualized representation.

7. Decoder:

- Similar to encoder but includes an additional **masked multi-head attention** (to prevent attending to future tokens during generation) and a **cross-attention** layer that attends to the encoder's output.

Diagram (describe as blocks):

```

Input → Embedding + Positional Encoding → [Encoder Stack] → Encoder Output
                                     ↓ (cross-attention)
Output (shifted right) → Embedding + Positional Encoding → [Decoder Stack] → Linear → Softmax → Output Probabilities
  
```

Significance: Transformers form the foundation of models like **BERT, GPT, and modern LLMs**, and outperform RNN/LSTM on many NLP tasks due to parallelization and better long-range dependency handling.

Q6(a) Explain the steps involved in Natural Language Processing. (7 marks) — Unit 6

Definition: Natural Language Processing (NLP) is a field of AI concerned with enabling computers to understand, interpret, and generate human language.

Steps/Pipeline of NLP:

1. **Lexical/Morphological Analysis:** Breaking text into tokens (words/sentences) — **tokenization**; identifying word structure (roots, prefixes/suffixes) via **stemming/lemmatization**. Example: "running" → "run"
2. **Syntactic Analysis (Parsing):** Analyzing grammatical structure of sentences using grammar rules, producing a parse tree. Includes **Part-of-Speech (POS) tagging** (labeling words as noun, verb, adjective, etc.). Example: "The cat sat" → [Noun Phrase: The cat] + [Verb Phrase: sat]
3. **Semantic Analysis:** Extracting the meaning of words and sentences, resolving word-sense ambiguity (e.g., "bank" = riverbank vs financial bank), building meaning representations.
4. **Discourse Integration:** Understanding how a sentence relates to preceding/following sentences — resolving pronoun references (coreference resolution), e.g., "John went to the store. **He** bought milk" → "He" = John.
5. **Pragmatic Analysis:** Interpreting meaning based on real-world context, intent, and knowledge beyond literal text — e.g., understanding sarcasm, implied requests ("Can you pass the salt?" = a request, not a literal question about ability).

Additional practical steps often included in an NLP pipeline:

- Stop-word removal
- Named Entity Recognition (NER)
- Word embeddings/vectorization (e.g., Word2Vec, TF-IDF)
- Model application (classification, translation, generation, etc.)

Together, these stages transform raw, unstructured text into a structured form that machine learning models can process for tasks like sentiment analysis, machine translation, chatbots, and information extraction.

Q6(b) Explain the concepts of LSTM for NLP. (8 marks) — Unit 6

Definition: Long Short-Term Memory (LSTM) is a special type of Recurrent Neural Network (RNN) architecture designed to learn long-term dependencies in sequential data, overcoming the **vanishing gradient problem** that plagues standard RNNs.

Why LSTM over standard RNN: Standard RNNs struggle to retain information over long sequences because gradients shrink exponentially during backpropagation through time. LSTM solves this using a **memory cell** and **gating mechanisms** that control what information to keep, forget, or output.

LSTM Cell Structure — Three Gates + Cell State:

1. **Forget Gate (ft):** Decides what information to discard from the cell state.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

2. **Input Gate (it):** Decides what new information to store in the cell state.

$$\begin{aligned} i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \\ \tilde{c}_t &= \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \end{aligned}$$

3. **Cell State Update (Ct):** Combines the forget and input gate results to update memory:

$$C_t = f_t * C_{t-1} + i_t * \tilde{c}_t$$

4. **Output Gate (ot):** Decides what part of the cell state to output as the hidden state:

$$\begin{aligned} o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \\ h_t &= o_t * \tanh(C_t) \end{aligned}$$

Diagram (describe as flow):

```

[xt, ht-1] → Forget Gate → decides what to erase from Ct-1
[xt, ht-1] → Input Gate → decides what new info to add
      ↓
    Cell State (Ct) updated (long-term memory, flows through time with minimal change)
      ↓
[xt, ht-1] → Output Gate → produces ht (short-term output/hidden state)
  
```

Application in NLP:



- **Language Modeling:** Predicting the next word based on long context.
- **Machine Translation:** Encoder-decoder LSTMs translate sentences between languages, retaining context across long sentences.
- **Sentiment Analysis:** Retaining sentiment-relevant information across an entire review/document.
- **Text Generation & Speech Recognition.**

LSTMs were the dominant NLP architecture before Transformers, precisely because their gating mechanism allowed them to "remember" relevant words/context across long sequences of text — something plain RNNs could not do effectively.

Q6 — OR — Describe the architecture of BERT.

Definition: BERT (Bidirectional Encoder Representations from Transformers) is a pre-trained language model developed by Google that uses only the **encoder stack** of the Transformer architecture, and is trained to understand context **bidirectionally** (looking at both left and right context simultaneously), unlike earlier models that read text only left-to-right.

Architecture:

- BERT is built from multiple stacked **Transformer Encoder layers** (BERT-Base: 12 layers, 12 attention heads, 110M parameters; BERT-Large: 24 layers, 16 attention heads, 340M parameters).
- Each encoder layer contains **Multi-Head Self-Attention + Feed-Forward Network**, with residual connections and layer normalization (identical to the standard Transformer encoder).

Input Representation: BERT's input embedding for each token is the sum of three embeddings:

1. **Token Embeddings:** WordPiece sub-word tokens.
2. **Segment Embeddings:** Distinguishes Sentence A vs Sentence B (for tasks involving sentence pairs).
3. **Positional Embeddings:** Encodes token position in the sequence.

Special tokens: [CLS] (added at the start, used for classification tasks) and [SEP] (separates two sentences).

Pre-training Tasks (what makes BERT bidirectional):

1. **Masked Language Model (MLM):** Randomly masks ~15% of tokens in the input, and the model must predict the masked word using context from **both** directions (left and right) — this is what enables true bidirectionality.
2. **Next Sentence Prediction (NSP):** Given two sentences, the model predicts whether the second sentence logically follows the first — helps BERT understand relationships between sentences.

Fine-tuning: After pre-training on large unlabeled text corpora, BERT can be fine-tuned with a small additional output layer for specific downstream tasks — question answering, sentiment analysis, named entity recognition, natural language inference — without needing to change the core architecture.

Significance: BERT's bidirectional context understanding significantly improved performance across almost all NLP benchmark tasks, and it remains a foundational model in modern NLP.

Q7. Write short notes on (Any Two): (2×5 marks)

a) Perceptron (Unit 5)

A Perceptron is the simplest type of artificial neural network unit, and the earliest model of a neural network, introduced by Frank Rosenblatt in 1958. It's a **binary linear classifier** that mimics a single biological neuron.

Structure:

- Takes multiple binary/real-valued inputs ($x_1, x_2, \dots, x_n$).



- Each input has an associated weight ($w_1, w_2, \dots, w_n$).
- Computes a weighted sum plus bias: $z = \sum(w_i \cdot x_i) + b$
- Passes the sum through an **activation function** (originally a step function) to produce a binary output:

$$\text{output} = 1 \text{ if } z \geq 0, \text{ else } 0$$

Learning Rule: Weights are updated iteratively based on the prediction error:

$$w_i = w_i + \eta(\text{target} - \text{output}) \times x_i$$

where η is the learning rate.

Limitation: A single perceptron can only solve **linearly separable** problems (e.g., it cannot learn XOR). This limitation led to the development of **Multi-Layer Perceptrons (MLPs)** with hidden layers and non-linear activation functions, capable of learning complex, non-linear decision boundaries via backpropagation.

b) Transfer Learning (Unit 5)

Transfer Learning is a machine learning technique where a model trained on one task (usually with a large dataset) is reused/adapted as the starting point for a model on a second, related task, instead of training a new model from scratch.

Why it's useful:

- Training deep networks from scratch requires huge labeled datasets and computation.
- Many tasks (e.g., image recognition) share common low-level features (edges, shapes, textures) that don't need to be re-learned for every new task.

Working Approach:

1. Take a model **pre-trained** on a large dataset (e.g., ResNet trained on ImageNet, or BERT pre-trained on large text corpora).
2. **Freeze** the early layers (which capture general features).
3. **Replace/retrain** the final layers (classifier head) on the new, smaller, task-specific dataset.
4. Optionally, **fine-tune** some or all layers with a small learning rate for better task-specific performance.

Example: A CNN trained on ImageNet (millions of general images) can be fine-tuned to classify medical X-ray images with only a few thousand labeled samples, since the pre-trained model already understands general visual features like edges and shapes.

Benefits: Saves training time, requires less labeled data, and often achieves better performance than training from scratch, especially when the target dataset is small.

c) Complications of Real Natural Language (Unit 6)

Natural language, unlike formal/programming languages, is inherently ambiguous and irregular, which creates several complications for NLP systems:

1. **Lexical Ambiguity:** A single word can have multiple meanings. Example: "Bank" (riverbank vs financial institution).
2. **Syntactic Ambiguity:** A sentence can be parsed in more than one grammatically valid way. Example: "I saw the man with the telescope" (who has the telescope — me or the man?).
3. **Semantic Ambiguity:** Even after parsing, the meaning of a sentence can remain unclear. Example: "Visiting relatives can be boring" (is it the act of visiting, or the relatives who visit, that's boring?).

4. **Anaphora/Coreference Resolution:** Determining what a pronoun refers to. Example: "The trophy doesn't fit in the suitcase because **it** is too big" — does "it" refer to the trophy or the suitcase?
5. **Pragmatic Ambiguity:** Meaning depends on context, tone, culture, or speaker intent (sarcasm, idioms). Example: "It's raining cats and dogs" (idiomatic, not literal).
6. **Ellipsis:** Parts of a sentence are omitted but understood from context. Example: "John likes tea, and Mary coffee" (implied "likes" before coffee).

These complications make it difficult for NLP systems to achieve fully accurate language understanding, and are a key reason why deep, context-aware models like Transformers and BERT (which use large amounts of context to disambiguate meaning) have significantly outperformed older rule-based/statistical NLP approaches.

End of Answer Sheet. Good luck with your exam!